PropOS 系统架构设计文档

> 版本: v1.1
>
> 日期: 2026-04-06
>
> 对应 **PRD**: v1.6
>
> 范围: Phase 1 全模块

目录

0. v1.6 对齐说明
1. 整体架构概览
    - 技术选型
    - 平台能力矩阵
2. 后端 Dart 服务目录结构
    - 核心分层原则
3. PostgreSQL 数据库 Schema
    - 3.1 枚举类型定义
    - 3.2 资产模块（M1）
    - 3.3 认证与用户模块
    - 3.4 租务与合同模块（M2）
    - 3.5 财务模块（M3）
    - 3.6 工单模块（M4）
    - 3.7 二房东穿透管理模块（M5）
    - 3.8 通用审计日志表
4. Flutter App 页面路由结构
    - 4.1 技术选型
    - 4.2 路由树定义
    - 4.3 路由守卫（角色路由映射）
5. RBAC 权限矩阵代码实现
    - 5.1 权限枚举定义
    - 5.2 角色-权限矩阵
    - 5.3 RBAC 中间件（Shelf）
    - 5.4 请求上下文注入（JWT 解析中间件）
    - 5.5 路由注册示例（Controller 层）
6. 二房东数据行级隔离 Repository 实现
    - 6.1 设计原则

6.2 RequestContext 结构
6.3 SubleaseRepository 行级隔离实现
6.4 审计日志自动记录
6.5 二房东账号生命周期管理
7. 关键数据流说明
7.1 账单自动生成流程
7.2 WALE 实时计算
7.3 KPI 自动打分（线性插值）
7.4 任务可靠性与人工补偿

---

0. v1.6 对齐说明

本版架构文档对齐 PRD v1.6，重点吸收以下约束：

1. Phase 1 以 Must 范围交付为优先，Should 和 Could 能力不阻塞首批上线。
2. 财务模型需支持部分收款、一次收多账单、发票状态与收款状态解耦。
3. KPI 在 Phase 1 定位为经营分析与试运行评分，不直接承接正式绩效。
4. 二房东门户需补充登录安全、会话失效、审核 SLA 与版本留痕。
5. 定时任务与消息触发需具备重试、失败列表与人工补偿能力。

> 交付边界: 架构设计优先保障资产、合同、账单、收款、工单、二房东填报六条核心链路闭环；复杂 KPI 排名、二房东 API 自动推送和增强分析作为可延后能力保留扩展点。

---

1. 整体架构概览

```
┌──────────────────────────────────────────────────────────────────────┐
│                              客户端层                                 │
│  ┌──────────────────────────┐  ┌──────────────────┐  ┌──────────────┐ │
│  │       Flutter App        │  │  Web 管理后台 │  │ 微信小程序 │ │
│  │ iOS / Android / 桌面端 │  │ (Flutter Web) │  │ (精简报修)  │ │
│  │ (macOS / Windows)        │  │                  │  │              │ │
│  └──────────┬───────────────┘  └────────┬─────────┘  └──────┬───────┘ │
└─────────────┼──────────────────────────┼──────────────────────┼────────┘
              │                          │                      │
              └──────────────────────────┴──────────────────────┘
                                         ▼ HTTPS / REST + JWT
┌──────────────────────────────────────────────────────────────────────┐
│                        Dart 后端服务（Shelf）                         │
│  ┌──────────┐  ┌──────────┐  ┌──────────┐  ┌──────────────┐          │
│  │   Auth   │  │ Assets   │  │ Contracts │  │   Finance    │          │
│  │ Module   │  │ Module   │  │ Module   │  │   Module     │          │
│  └──────────┘  └──────────┘  └──────────┘  └──────────────┘          │
│                                                                       │
│  ┌──────────┐  ┌──────────┐  ┌──────────────────────────┐            │
```

```
│   │ Workorder│   │ SubLease  │   │          KPI Module          │   │
│   │  Module  │   │  Module   │   │                              │   │
│   └──────────┘   └───────────┘   └──────────────────────────────┘   │
│                                                                      │
│                          ▲                                           │
│                          │                                           │
│                 ┌────────┴──────────┐                                │
│                 │   RBAC 中间件     │  Audit 日志中间件     │
│                 └───────────────────┘                                │
└──────────────────────────────────────────────────────────────────────┘
                                  ▼
┌──────────────────────────────────────────────────────────────────────┐
│                          数据层                  │                   │
│   ┌──────────────────────┐   ┌──────────────────────────────────┐   │
│   │   PostgreSQL 主库    │   │   文件存储（S3 兼容 / 本地）     │ │
│   │  (业务数据 + 审计日志) │   │  (CAD 平面图、合同 PDF、照片)     │ │
│   └──────────────────────┘   └──────────────────────────────────┘   │
└──────────────────────────────────────────────────────────────────────┘
```

技术选型

层次	技术	说明
后端框架	Dart + Shelf	HTTP 服务器框架，中间件管道模式
ORM / 查询	`postgres` 包（原生 SQL）	避免 ORM 魔法，行级隔离逻辑显式可控
数据模型	`freezed` + `json_serializable`	不可变数据类，自动序列化
认证	JWT（`dart_jsonwebtoken`）	Claims 携带 role + sub*landlord*scope
数据库	PostgreSQL 15+	行级安全 + GIN 索引
文件转换	`oc2svg` CLI + 后端调度	.dwg → SVG/PNG 楼层平面图
APP	Flutter 3.x（`go_router`）	iOS / Android / macOS / Windows / Web 五端复用；桌面端不含 QR 扫码与 FCM 推送，其余功能完整

平台能力矩阵

功能	iOS	Android	macOS	Windows	Flutter Web
全部业务页面（Dashboard、资产、租务、财务）	✅	✅	✅	✅	✅
QR 扫码报修（`mobile_scanner`）	✅	✅	❌ → 手动填报	❌ → 手动填报	❌ → 手动填报
FCM 推送通知（`firebase_messaging`）	✅	✅	⚠️ 有限	❌	❌
应用内通知 + 轮询（桌面/Web 替代方案）	—	—	✅	✅	✅
文件选择 / Excel 导入（`file_picker`）	✅	✅	✅	✅	✅
相机拍照上传（`image_picker`）	✅	✅	✅	⚠️	⚠️
SVG 楼层热区图	✅	✅	✅	✅	✅

> **扫码替代方案**：桌面端 `QrScanPage` 降级为 `WorkOrderFormPage（手动输入楼栋/楼层/单元号）`。
>
> **通知替代方案**：桌面端 / Web 使用应用内 Badge 角标 + 30 秒轮询 `/api/alerts/unread`，不依赖 FCM。
>
> **平台判断统一封装**：所有 `Platform.isXxx` / `kIsWeb` 判断集中在 `lib/shared/platform_utils.dart`，页面层不散落平台检测代码。

```
// lib/shared/platform_utils.dart
import 'dart:io';
import 'package:flutter/foundation.dart';

class PlatformUtils {
  static bool get isMobile =>
      !kIsWeb && (Platform.isIOS || Platform.isAndroid);

  static bool get isDesktop =>
```

```
      !kIsWeb && (Platform.isMacOS || Platform.isWindows ||
Platform.isLinux);

  static bool get supportsCamera => isMobile;

  static bool get supportsQrScan => isMobile;

  static bool get supportsPushNotification => isMobile;
}
```

---

2. 后端 Dart 服务目录结构

```
backend/
├── packages/                          # 独立 Dart package（零外部依赖，纯计算
库）
│   ├── rent_escalation_engine/        # 租金递增计算引擎
│   │   ├── lib/src/
│   │   │   ├── escalation_rule.dart   # RentEscalationRule sealed class（6种
子类型）
│   │   │   └── rent_calculator.dart   # RentCalculator.compute(rules, date)
→ Money
│   │   ├── test/
│   │   │   └── rent_escalation_test.dart  # 覆盖：固定/阶梯/CPI/混合分段
│   │   └── pubspec.yaml               # name: rent_escalation_engine（无业务
依赖）
│   └── kpi_scorer/                    # KPI 线性插值打分引擎
│       ├── lib/src/
│       │   ├── kpi_metric.dart        # KpiMetric（指标定义 + 满分/及格/不及格阈
值）
│       │   └── kpi_scorer.dart        # KpiScorer.score(metric, actual) →
0~100
│       ├── test/
│       │   └── kpi_scorer_test.dart   # 覆盖：满分/及格/零分/边界值
│       └── pubspec.yaml               # name: kpi_scorer（无业务依赖）
│
├── bin/
│   └── server.dart                    # 入口：启动 Shelf HTTP 服务
├── lib/
│   ├── config/
│   │   ├── app_config.dart            # 环境变量读取（DB_URL、JWT_SECRET 等）
│   │   └── database.dart             # PostgreSQL 连接池初始化
│   │
│   ├── jobs/
│   │   ├── job_runner.dart            # 定时任务统一入口
│   │   ├── job_execution_log.dart     # 任务执行日志与失败列表
│   │   └── retry_scheduler.dart       # 失败重试与人工补偿任务调度
│   │
│   ├── core/
│   │   ├── middleware/
```

```
│   │   │   ├── auth_middleware.dart   # JWT 验证，注入 RequestContext
│   │   │   ├── rbac_middleware.dart   # RBAC 权限检查（见第5节）
│   │   │   └── audit_middleware.dart  # 操作审计日志写入
│   │   ├── errors/
│   │   │   ├── app_exception.dart     # 统一异常基类
│   │   │   └── error_handler.dart     # 全局异常 → HTTP 状态码映射
│   │   ├── pagination.dart            # 分页参数解析与响应包装
│   │   └── request_context.dart      # 请求上下文（用户ID、角色、二房东范围）
│   │
│   ├── modules/
│   │   │
│   │   ├── auth/                      # 认证模块
│   │   │   ├── models/
│   │   │   │   ├── user.dart          # @freezed User
│   │   │   │   └── role.dart         # Role 枚举
│   │   │   ├── repositories/
│   │   │   │   └── user_repository.dart
│   │   │   ├── services/
│   │   │   │   ├── auth_service.dart  # 登录、刷新 Token、登录失败锁定
│   │   │   │   └── token_service.dart # JWT 签发与验证
│   │   │   └── controllers/
│   │   │       └── auth_controller.dart
│   │   │
│   │   ├── assets/                    # 模块1：资产与空间
│   │   │   ├── models/
│   │   │   │   ├── building.dart      # @freezed Building
│   │   │   │   ├── floor.dart        # @freezed Floor（含 SVG 图层路径）
│   │   │   │   ├── unit.dart         # @freezed Unit（含三业态扩展字段联合体）
│   │   │   │   └── renovation_record.dart
│   │   │   ├── repositories/
│   │   │   │   ├── building_repository.dart
│   │   │   │   ├── floor_repository.dart
│   │   │   │   └── unit_repository.dart
│   │   │   ├── services/
│   │   │   │   ├── unit_service.dart  # 业务逻辑（状态计算、色块聚合）
│   │   │   │   ├── cad_import_service.dart  # .dwg → SVG/PNG 转换调度
│   │   │   │   └── unit_import_service.dart # Excel 批量导入 639 套
│   │   │   └── controllers/
│   │   │       ├── building_controller.dart
│   │   │       ├── floor_controller.dart
│   │   │       └── unit_controller.dart
│   │   │
│   │   ├── contracts/                 # 模块2：租务与合同
│   │   │   ├── models/
│   │   │   │   ├── tenant.dart        # @freezed Tenant（证件号标注加密）
│   │   │   │   ├── contract.dart     # @freezed Contract（含状态机枚举）
│   │   │   │   ├── contract_status.dart  # ContractStatus 枚举
│   │   │   │   └── alert.dart        # Alert 预警记录
│   │   │   │   # ↑ RentEscalationRule 已移入 packages/rent_escalation_engine
│   │   │   ├── repositories/
│   │   │   │   ├── tenant_repository.dart
```

```
│   │   │   │   ├── contract_repository.dart
│   │   │   │   └── alert_repository.dart
│   │   │   ├── services/
│   │   │   │   ├── contract_service.dart     # 合同 CRUD + 状态机转换
│   │   │   │   ├── wale_service.dart         # WALE 计算（调用 RentCalculator
获取年化租金）
│   │   │   │   ├── rent_escalation_service.dart  # 持久化递增规则配置，调用
rent_escalation_engine
│   │   │   │   └── alert_service.dart        # 预警触发调度（定时任务钩子）
│   │   │   └── controllers/
│   │   │       ├── tenant_controller.dart
│   │   │       ├── contract_controller.dart
│   │   │       └── wale_controller.dart
│   │   │
│   │   ├── finance/                   # 模块3：财务
│   │   │   ├── models/
│   │   │   │   ├── invoice.dart       # @freezed Invoice（账单）
│   │   │   │   ├── payment.dart      # @freezed Payment（收款主记录）
│   │   │   │   ├── payment_allocation.dart # @freezed PaymentAllocation（核
销分配）
│   │   │   │   ├── expense.dart      # @freezed Expense（运营支出）
│   │   │   │   ├── kpi_scheme.dart   # @freezed KpiScheme（KPI 试运行方案配
置）
│   │   │   │   └── kpi_score.dart    # @freezed KpiScore（评分快照，持久化用）
│   │   │   ├── repositories/
│   │   │   │   ├── invoice_repository.dart
│   │   │   │   ├── payment_repository.dart
│   │   │   │   ├── payment_allocation_repository.dart
│   │   │   │   ├── expense_repository.dart
│   │   │   │   └── kpi_repository.dart
│   │   │   ├── services/
│   │   │   │   ├── invoice_service.dart      # 自动账单生成（调用
RentCalculator）
│   │   │   │   ├── receivable_service.dart   # 核销分配、部分收款、跨账单收款
│   │   │   │   ├── noi_service.dart          # NOI 实时计算（EGI - OpEx）
│   │   │   │   └── kpi_service.dart          # 数据聚合 + 调用 KpiScorer 打分，
结果写库（试运行）
│   │   │   └── controllers/
│   │   │       ├── invoice_controller.dart
│   │   │       ├── payment_controller.dart
│   │   │       ├── noi_controller.dart
│   │   │       └── kpi_controller.dart
│   │   │
│   │   ├── workorders/                # 模块4：工单
│   │   │   ├── models/
│   │   │   │   ├── work_order.dart    # @freezed WorkOrder（含状态机枚举）
│   │   │   │   ├── work_order_status.dart
│   │   │   │   └── supplier.dart     # @freezed Supplier（供应商）
│   │   │   ├── repositories/
│   │   │   │   ├── work_order_repository.dart
│   │   │   │   └── supplier_repository.dart
```

```
│   │   │   ├── services/
│   │   │   │   ├── work_order_service.dart   # 状态机转换 + 成本归口
│   │   │   │   └── push_service.dart         # APNs/FCM 推送封装
│   │   │   └── controllers/
│   │   │       └── work_order_controller.dart
│   │   │
│   │   └── subleases/                  # 模块5：二房东穿透
│   │       ├── models/
│   │       │   ├── sublease.dart       # @freezed SubLease（子租赁）
│   │       │   └── sublease_status.dart  # SubLeaseStatus 枚举
│   │       ├── repositories/
│   │       │   └── sublease_repository.dart  # 行级隔离实现（见第6节）
│   │       ├── services/
│   │       │   ├── sublease_service.dart     # 审核流、填报提醒、版本留痕
│   │       │   ├── portal_session_service.dart # 外部门户会话失效与单点登录控制
│   │       │   └── sublease_import_service.dart  # Excel 批量上传
│   │       └── controllers/
│   │           └── sublease_controller.dart
│   │
│   ├── router/
│   │   └── app_router.dart             # 统一路由注册（mount 各模块路由）
│   │
│   └── shared/
│       ├── encryption.dart             # AES-256 加解密（证件号、手机号）
│       ├── task_outbox.dart            # 消息发送结果留痕与失败补偿
│       └── validators.dart             # 通用校验工具
│
├── test/
│   ├── unit/
│   │   ├── wale_service_test.dart      # WALE 计算单元测试（调用 RentCalculator
mock）
│   │   └── noi_service_test.dart       # NOI 计算单元测试
│   │   # ↑ 递增规则/KPI 打分的纯函数测试已移入各自 package 的 test/ 目录
│   └── integration/
│       └── contract_lifecycle_test.dart
│
├── migrations/                         # 数据库迁移脚本（按版本顺序执行）
│   ├── 001_create_users.sql
│   ├── 002_create_assets.sql
│   ├── 003_create_contracts.sql
│   ├── 004_create_finance.sql
│   ├── 005_create_workorders.sql
│   └── 006_create_subleases.sql
│
└── pubspec.yaml                        # path 依赖本地两个 package
```

`pubspec.yaml` 本地依赖声明：

```
dependencies:
  rent_escalation_engine:
    path: ./packages/rent_escalation_engine
  kpi_scorer:
```

```
    path: ./packages/kpi_scorer
```

核心分层原则

层	职责	禁止
`packages/`	零副作用纯计算库，无 IO、无 DB、无 HTTP	不依赖业务模块
`controllers/`	HTTP 请求解析、参数校验、响应序列化	不含业务逻辑
`services/`	业务规则、状态机、数据编排；调用 `packages/` 中的计算函数	不含 SQL
`repositories/`	所有 SQL 查询，行级隔离在此层强制	不含业务规则
`models/`	`freezed` 不可变数据类；含实体模型与 Command 对象	不含副作用

Controller → Service 传递约定（消除 JSON 与内部模型的歧义）：

- Controller 负责将 HTTP 请求 JSON body 解析为强类型 **Command 对象**（如 `CreateContractCommand`、`UpdateUnitCommand`），再传入 Service；**Service 方法签名只接受强类型参数，禁止接受`Map<String, dynamic>`**
- 格式校验（字段存在性、类型转换、枚举合法性）在 Controller 层完成，校验失败直接抛 `AppException('INVALID_REQUEST', ..., 400)`
- 业务校验（如合同结束日必须晚于起始日、单元不可重复签约）属于 Service 层，Controller 不做业务判断
- Command 对象定义在所属模块的 `models/` 目录下（与实体模型并列），命名规则：`动词 + 资源名 + Command`，例如 `CreateContractCommand`、`WriteOffInvoiceCommand`
- GET 请求的 Query 参数（分页、筛选）直接以具名参数传入 Service，无需封装 Command 对象

---

3. PostgreSQL 数据库 Schema

3.1 枚举类型定义

```
-- 三业态枚举
CREATE TYPE property_type AS ENUM ('office', 'retail', 'apartment');

-- 单元出租状态
CREATE TYPE unit_status AS ENUM ('vacant', 'leased', 'expiring_soon',
'non_leasable');
```

```
-- 合同状态机
CREATE TYPE contract_status AS ENUM (
  'quoting', 'pending_sign', 'active', 'expiring_soon', 'expired',
'renewed', 'terminated'
);

-- 租金递增类型
CREATE TYPE escalation_type AS ENUM (
  'fixed_percent', 'fixed_amount', 'stepped', 'cpi_linked', 'every_n_years',
'post_free_rent'
);

-- 账单状态
CREATE TYPE invoice_status AS ENUM ('pending', 'paid', 'overdue',
'cancelled', 'waived');

-- 工单状态
CREATE TYPE work_order_status AS ENUM (
  'submitted', 'approved', 'in_progress', 'pending_acceptance', 'completed',
'rejected', 'on_hold'
);

-- 工单紧急程度
CREATE TYPE urgency_level AS ENUM ('normal', 'urgent', 'critical');

-- 子租赁入住状态
CREATE TYPE occupancy_status AS ENUM (
  'occupied', 'signed_not_moved_in', 'moved_out', 'vacant'
);

-- 子租赁审核状态
CREATE TYPE review_status AS ENUM ('draft', 'pending_review', 'approved',
'rejected');

-- 用户角色
CREATE TYPE user_role AS ENUM (
  'super_admin', 'ops_manager', 'leasing_agent', 'finance_staff',
'frontline', 'sub_landlord'
);

-- KPI 评估周期
CREATE TYPE kpi_period_type AS ENUM ('monthly', 'quarterly', 'yearly');
```

> **v1.6 口径说明**: KPI 相关表继续保留，但默认用于试运行评分和经营分析，不作为正式绩效的强制结算依据。

3.2 资产模块（M1）

```
-- 楼栋表
CREATE TABLE buildings (
```

```
  id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name          VARCHAR(100) NOT NULL,             -- 'A座', '商铺区', '公寓
楼'
  property_type property_type NOT NULL,
  total_floors  SMALLINT NOT NULL,
  gfa           NUMERIC(10, 2) NOT NULL,           -- 总建筑面积（m²）
  nla           NUMERIC(10, 2),                    -- 净可租面积（m²）
  address       TEXT,
  created_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at    TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- 楼层表
CREATE TABLE floors (
  id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  building_id   UUID NOT NULL REFERENCES buildings(id) ON DELETE CASCADE,
  floor_number  SMALLINT NOT NULL,                 -- 负数表示地下层
  floor_name    VARCHAR(50),                       -- '1F', 'B1' 等展示名
  svg_path      TEXT,                              -- 转换后 SVG 存储路径
  png_path      TEXT,                              -- 备用 PNG 路径
  created_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  UNIQUE (building_id, floor_number)
);

-- 单元表（核心资产底座）
CREATE TABLE units (
  id                  UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  floor_id            UUID NOT NULL REFERENCES floors(id) ON DELETE CASCADE,
  building_id         UUID NOT NULL REFERENCES buildings(id),  -- 冗余方便查询
  unit_no             VARCHAR(50) NOT NULL,
  property_type       property_type NOT NULL,
  gross_area          NUMERIC(8, 2) NOT NULL,       -- 建筑面积（m²）
  net_area            NUMERIC(8, 2),                -- 套内面积（m²）
  floor_height        NUMERIC(4, 2),                -- 层高（m）
  orientation         VARCHAR(20),                  -- 朝向
  decoration_status   VARCHAR(50),                  -- 装修状态
  status              unit_status NOT NULL DEFAULT 'vacant',
  svg_hotzone_coords  JSONB,                        -- 热区多边形坐标 [{x,y}]

  -- 写字楼扩展字段（property_type = 'office' 时有值）
  workstation_count   SMALLINT,
  partition_count     SMALLINT,

  -- 商铺扩展字段（property_type = 'retail' 时有值）
  frontage_width      NUMERIC(6, 2),               -- 门面宽度（m）
  street_facing       BOOLEAN,                      -- 是否临街
  retail_floor_height NUMERIC(4, 2),               -- 商铺层高（m）

  -- 公寓扩展字段（property_type = 'apartment' 时有值）
  bedroom_count       SMALLINT,
  private_bathroom    BOOLEAN,
```

```
  created_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  UNIQUE (floor_id, unit_no)
);

CREATE INDEX idx_units_building_id ON units(building_id);
CREATE INDEX idx_units_property_type ON units(property_type);
CREATE INDEX idx_units_status ON units(status);

-- 改造记录表
CREATE TABLE renovation_records (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  unit_id         UUID NOT NULL REFERENCES units(id) ON DELETE CASCADE,
  renovation_type VARCHAR(100) NOT NULL,
  start_date      DATE NOT NULL,
  end_date        DATE,
  cost            NUMERIC(12, 2),                  -- 施工造价（元）
  contractor      VARCHAR(200),
  notes           TEXT,
  created_by      UUID NOT NULL REFERENCES users(id),
  created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- 改造照片表
CREATE TABLE renovation_photos (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  renovation_id     UUID NOT NULL REFERENCES renovation_records(id) ON
DELETE CASCADE,
  photo_url         TEXT NOT NULL,
  photo_type        VARCHAR(20) NOT NULL CHECK (photo_type IN ('before',
'after')),
  uploaded_at       TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

3.3 认证与用户模块

```
-- 用户表
CREATE TABLE users (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  email           VARCHAR(255) NOT NULL UNIQUE,
  password_hash   VARCHAR(255) NOT NULL,            -- bcrypt hash
  full_name       VARCHAR(100) NOT NULL,
  role            user_role NOT NULL,
  is_active       BOOLEAN NOT NULL DEFAULT TRUE,
  -- 二房东专用字段：绑定主合同（为 NULL 表示非二房东角色）
  master_contract_id UUID REFERENCES contracts(id), -- 行级隔离锚点
  created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at      TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_users_role ON users(role);
```

```
CREATE INDEX idx_users_master_contract ON users(master_contract_id)
  WHERE master_contract_id IS NOT NULL;

-- 刷新令牌表（支持多设备登录 + 强制下线）
CREATE TABLE refresh_tokens (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id     UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  token_hash  VARCHAR(255) NOT NULL UNIQUE,
  device_info TEXT,
  expires_at  TIMESTAMPTZ NOT NULL,
  revoked     BOOLEAN NOT NULL DEFAULT FALSE,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

3.4 租务与合同模块（M2）

```
-- 租客表
CREATE TABLE tenants (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_type     VARCHAR(20) NOT NULL CHECK (tenant_type IN ('company',
'individual')),
  name            VARCHAR(200) NOT NULL,            -- 企业名或个人姓名
  -- 证件号加密存储（AES-256-GCM），API 层默认脱敏展示后4位
  cert_no_encrypted    BYTEA,                       -- 加密存储
  cert_no_hint         VARCHAR(10),                 -- 脱敏展示（后4位）
  unified_social_code  VARCHAR(50),                 -- 统一社会信用代码（企业）
  contact_name    VARCHAR(100),
  -- 手机号加密存储
  phone_encrypted BYTEA,
  phone_hint      VARCHAR(10),                      -- 后4位
  email           VARCHAR(255),
  emergency_contact     VARCHAR(100),
  emergency_phone_encrypted BYTEA,
  credit_rating   CHAR(1) CHECK (credit_rating IN ('A', 'B', 'C')),
  overdue_count   SMALLINT NOT NULL DEFAULT 0,      -- 历史逾期次数（系统自动累
计）
  created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at      TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- 合同表
CREATE TABLE contracts (
  id                  UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  contract_no         VARCHAR(100) NOT NULL UNIQUE,
  tenant_id           UUID NOT NULL REFERENCES tenants(id),
  is_sub_landlord     BOOLEAN NOT NULL DEFAULT FALSE,  -- 是否为二房东主合同
  status              contract_status NOT NULL DEFAULT 'quoting',
  property_type       property_type NOT NULL,          -- 冗余 便于 WALE 业态分
组
  start_date          DATE NOT NULL,
```

```
  end_date            DATE NOT NULL,
  free_rent_days      SMALLINT NOT NULL DEFAULT 0,     -- 免租天数
  fit_out_days        SMALLINT NOT NULL DEFAULT 0,     -- 装修期天数（免收费用，
不计逾期）
  base_rent           NUMERIC(12, 2) NOT NULL,         -- 签约基准月租金（元）
  deposit             NUMERIC(12, 2),                  -- 押金（元）
  payment_cycle_days  SMALLINT NOT NULL DEFAULT 30,    -- 付款周期（天），通常
30/90/180
  -- 商铺营业额分成（property_type = 'retail' 时有效）
  turnover_rent_pct   NUMERIC(5, 4),                   -- 分成比例（0.05 =
5%）
  min_rent_guarantee  NUMERIC(12, 2),                  -- 保底租金（元/月）
  -- 续签关联
  parent_contract_id  UUID REFERENCES contracts(id),   -- 续签时关联原合同
  -- 附件
  pdf_url             TEXT,
  notes               TEXT,
  created_by          UUID NOT NULL REFERENCES users(id),
  created_at          TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at          TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_contracts_tenant_id ON contracts(tenant_id);
CREATE INDEX idx_contracts_status ON contracts(status);
CREATE INDEX idx_contracts_end_date ON contracts(end_date);
CREATE INDEX idx_contracts_property_type ON contracts(property_type);

-- 合同-单元关联表（一份合同可覆盖多个单元，如二房东整层包租）
CREATE TABLE contract_units (
  contract_id UUID NOT NULL REFERENCES contracts(id) ON DELETE CASCADE,
  unit_id     UUID NOT NULL REFERENCES units(id),
  PRIMARY KEY (contract_id, unit_id)
);

-- 租金递增规则阶段表（一份合同多个阶段）
CREATE TABLE rent_escalation_stages (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  contract_id     UUID NOT NULL REFERENCES contracts(id) ON DELETE CASCADE,
  stage_order     SMALLINT NOT NULL,               -- 阶段序号（1, 2, 3...）
  stage_start     DATE NOT NULL,
  stage_end       DATE,
  escalation_type escalation_type NOT NULL,
  -- 参数（根据类型约定字段语义）
  percent_rate    NUMERIC(6, 4),                   -- 固定比例（0.05 = 5%）
  fixed_amount    NUMERIC(10, 2),                  -- 固定金额递增（元/m²）
  n_years         SMALLINT,                        -- 每 N 年递增一次
  stepped_table   JSONB,                           -- 阶梯表
[{from_year,to_year,rent}]
  cpi_year        SMALLINT,                        -- CPI 挂钩年份
  cpi_rate        NUMERIC(6, 4),                   -- 手工录入的 CPI 涨幅
  base_rent_override NUMERIC(12, 2),               -- 免租后基准价（override）
```

```
  UNIQUE (contract_id, stage_order)
);

-- 递增规则模板表
CREATE TABLE escalation_templates (
  id               UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name             VARCHAR(200) NOT NULL,
  property_type    property_type,                    -- NULL 表示通用模板
  is_default       BOOLEAN NOT NULL DEFAULT FALSE,
  stages_config    JSONB NOT NULL,                   -- 模板阶段配置 JSON
  created_by       UUID NOT NULL REFERENCES users(id),
  created_at       TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- 预警记录表
CREATE TABLE alerts (
  id               UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  alert_type       VARCHAR(50) NOT NULL,             -- 'expiry_90d',
'overdue_1d' 等
  contract_id      UUID REFERENCES contracts(id),
  invoice_id       UUID REFERENCES invoices(id),
  triggered_at     TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  is_read          BOOLEAN NOT NULL DEFAULT FALSE,
  target_roles     user_role[] NOT NULL               -- 接收角色数组
);

CREATE INDEX idx_alerts_contract ON alerts(contract_id);
CREATE INDEX idx_alerts_triggered ON alerts(triggered_at DESC);
```

3.5 财务模块（M3）

```
-- 账单表
CREATE TABLE invoices (
  id               UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  invoice_no       VARCHAR(100) NOT NULL UNIQUE,
  contract_id      UUID NOT NULL REFERENCES contracts(id),
  unit_id          UUID REFERENCES units(id),        -- 费项归口单元
  invoice_type     VARCHAR(50) NOT NULL,             -- 'rent', 'mgmt_fee',
'utility', 'parking'
  period_start     DATE NOT NULL,
  period_end       DATE NOT NULL,
  amount           NUMERIC(12, 2) NOT NULL,          -- 应收金额（元）
  tax_rate         NUMERIC(5, 4) NOT NULL DEFAULT 0,
  status           invoice_status NOT NULL DEFAULT 'pending',
  due_date         DATE NOT NULL,
  fapiao_no        VARCHAR(100),                     -- 发票号
  fapiao_status    VARCHAR(20) DEFAULT 'not_issued', -- 'not_issued',
'issued'
  notes            TEXT,
  created_at       TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at       TIMESTAMPTZ NOT NULL DEFAULT NOW()
```

```
);

CREATE INDEX idx_invoices_contract_id ON invoices(contract_id);
CREATE INDEX idx_invoices_status ON invoices(status);
CREATE INDEX idx_invoices_due_date ON invoices(due_date);
CREATE INDEX idx_invoices_period ON invoices(period_start, period_end);

-- 收款核销表
CREATE TABLE payments (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  invoice_id      UUID NOT NULL REFERENCES invoices(id),
  amount_paid     NUMERIC(12, 2) NOT NULL,
  payment_date    DATE NOT NULL,
  payment_method  VARCHAR(50),                      -- 'bank_transfer',
'cash' 等
  bank_ref        VARCHAR(200),                     -- 银行流水号
  verified_by     UUID NOT NULL REFERENCES users(id),
  created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- 运营支出表
CREATE TABLE expenses (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  expense_type    VARCHAR(100) NOT NULL,            -- 'utility',
'cleaning', 'repair', 'insurance', 'tax'
  building_id     UUID REFERENCES buildings(id),
  unit_id         UUID REFERENCES units(id),        -- 精确到单元则填
  floor_id        UUID REFERENCES floors(id),       -- 精确到楼层则填
  amount          NUMERIC(12, 2) NOT NULL,
  expense_date    DATE NOT NULL,
  description     TEXT,
  work_order_id   UUID REFERENCES work_orders(id),  -- 工单关联（维修费自动归
入）
  created_by      UUID NOT NULL REFERENCES users(id),
  created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_expenses_building ON expenses(building_id);
CREATE INDEX idx_expenses_date ON expenses(expense_date);

-- KPI 方案表
CREATE TABLE kpi_schemes (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name            VARCHAR(200) NOT NULL,
  period_type     kpi_period_type NOT NULL,
  valid_from      DATE NOT NULL,
  valid_to        DATE,
  indicators      JSONB NOT NULL,
  -- 格式:
[{"code":"K01","weight":0.2,"full_score_threshold":0.95,"pass_threshold":0.8
,"enabled":true}]
  created_by      UUID NOT NULL REFERENCES users(id),
  created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
```

```
  updated_at      TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- KPI 方案-用户绑定表
CREATE TABLE kpi_scheme_targets (
  scheme_id   UUID NOT NULL REFERENCES kpi_schemes(id) ON DELETE CASCADE,
  user_id     UUID REFERENCES users(id),
  department  VARCHAR(100),
  PRIMARY KEY (scheme_id, COALESCE(user_id::TEXT, department))
);

-- KPI 打分快照表（定期计算后持久化，避免重复计算）
CREATE TABLE kpi_scores (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  scheme_id       UUID NOT NULL REFERENCES kpi_schemes(id),
  user_id         UUID REFERENCES users(id),
  department      VARCHAR(100),
  period_start    DATE NOT NULL,
  period_end      DATE NOT NULL,
  total_score     NUMERIC(5, 2) NOT NULL,
  indicator_scores JSONB NOT NULL,                  --
{"K01":95.0,"K02":88.5,...}
  calculated_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  UNIQUE (scheme_id, COALESCE(user_id::TEXT, ''), period_start, period_end)
);
```

3.6 工单模块（M4）

```
-- 工单表
CREATE TABLE work_orders (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  order_no        VARCHAR(100) NOT NULL UNIQUE,
  building_id     UUID NOT NULL REFERENCES buildings(id),
  floor_id        UUID REFERENCES floors(id),
  unit_id         UUID REFERENCES units(id),
  reported_by     UUID NOT NULL REFERENCES users(id),
  assigned_to     UUID REFERENCES users(id),
  status          work_order_status NOT NULL DEFAULT 'submitted',
  urgency         urgency_level NOT NULL DEFAULT 'normal',
  issue_type      VARCHAR(100) NOT NULL,            -- '水电', '空调', '门窗',
'公共区域' 等
  description     TEXT NOT NULL,
  material_cost   NUMERIC(10, 2) DEFAULT 0,
  labor_cost      NUMERIC(10, 2) DEFAULT 0,
  supplier_id     UUID REFERENCES suppliers(id),
  submitted_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  approved_at     TIMESTAMPTZ,
  completed_at    TIMESTAMPTZ,
  notes           TEXT,
  created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at      TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

```
CREATE INDEX idx_work_orders_status ON work_orders(status);
CREATE INDEX idx_work_orders_building ON work_orders(building_id);
CREATE INDEX idx_work_orders_assigned ON work_orders(assigned_to);

-- 工单照片表
CREATE TABLE work_order_photos (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  order_id    UUID NOT NULL REFERENCES work_orders(id) ON DELETE CASCADE,
  photo_url   TEXT NOT NULL,
  photo_type  VARCHAR(20) NOT NULL CHECK (photo_type IN ('issue',
'completion')),
  uploaded_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- 供应商表
CREATE TABLE suppliers (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name        VARCHAR(200) NOT NULL,
  category    VARCHAR(100) NOT NULL,                -- '水电', '空调', '电梯'
等
  contact     VARCHAR(100),
  phone_hint  VARCHAR(10),                          -- 后4位脱敏
  phone_encrypted BYTEA,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

3.7 二房东穿透管理模块（M5）

```
-- 子租赁表（行级隔离核心表）
CREATE TABLE subleases (
  id                  UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  master_contract_id  UUID NOT NULL REFERENCES contracts(id),  -- 行级隔离锚点
  unit_id             UUID NOT NULL REFERENCES units(id),
  -- 终端租客信息
  sub_tenant_name     VARCHAR(200) NOT NULL,
  sub_tenant_type     VARCHAR(20) NOT NULL CHECK (sub_tenant_type IN
('company', 'individual')),
  contact_name        VARCHAR(100),
  contact_phone_encrypted BYTEA,
  contact_phone_hint  VARCHAR(10),
  -- 证件号加密存储（建议填写）
  cert_no_encrypted   BYTEA,
  cert_no_hint        VARCHAR(10),
  -- 租期
  start_date          DATE NOT NULL,
  end_date            DATE NOT NULL,
  -- 实际租金
  monthly_rent        NUMERIC(12, 2) NOT NULL,      -- 终端租客支付给二房东的月
租金
```

```
  rent_per_sqm        NUMERIC(8, 2) GENERATED ALWAYS AS
                        (monthly_rent / NULLIF(
                          (SELECT net_area FROM units WHERE id = unit_id), 0
                        )) STORED,                  -- 系统自动反算单价
  -- 入住状态
  occupancy_status    occupancy_status NOT NULL DEFAULT 'vacant',
  occupant_count      SMALLINT,                     -- 公寓适用
  notes               TEXT,
  -- 审核状态
  review_status       review_status NOT NULL DEFAULT 'draft',
  reviewed_by         UUID REFERENCES users(id),
  reviewed_at         TIMESTAMPTZ,
  reject_reason       TEXT,
  -- 元数据
  submitted_by        UUID NOT NULL REFERENCES users(id),  -- 二房东用户ID
  created_at          TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at          TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  -- 同一单元不可同时存在两条"在租"子租赁
  CONSTRAINT uq_active_sublease UNIQUE (unit_id, review_status)
    DEFERRABLE INITIALLY DEFERRED
);

CREATE INDEX idx_subleases_master_contract ON subleases(master_contract_id);
CREATE INDEX idx_subleases_unit ON subleases(unit_id);
CREATE INDEX idx_subleases_review_status ON subleases(review_status);

-- 子租赁变更审计日志表
CREATE TABLE sublease_audit_logs (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  sublease_id     UUID NOT NULL REFERENCES subleases(id),
  action          VARCHAR(50) NOT NULL,             -- 'create', 'update',
'submit', 'approve', 'reject'
  operator_id     UUID NOT NULL REFERENCES users(id),
  old_value       JSONB,
  new_value       JSONB,
  ip_address      INET,
  operated_at     TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_sublease_audit_sublease ON
sublease_audit_logs(sublease_id);
CREATE INDEX idx_sublease_audit_operated ON sublease_audit_logs(operated_at
DESC);
```

3.8 通用审计日志表

```
-- 全系统操作审计日志（合同变更、账单核销、权限变更记录在此）
CREATE TABLE audit_logs (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  operator_id UUID NOT NULL REFERENCES users(id),
```

```
  module      VARCHAR(50) NOT NULL,                -- 'contracts',
'finance', 'auth'
  action      VARCHAR(100) NOT NULL,
  entity_type VARCHAR(50) NOT NULL,
  entity_id   UUID,
  old_value   JSONB,
  new_value   JSONB,
  ip_address  INET,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_audit_logs_operator ON audit_logs(operator_id);
CREATE INDEX idx_audit_logs_entity ON audit_logs(entity_type, entity_id);
CREATE INDEX idx_audit_logs_created ON audit_logs(created_at DESC);
```

---

4. Flutter App 页面路由结构

4.1 技术选型

- 路由库：`go_router 14.x`
- 状态管理：`flutter_bloc`
- 鉴权守卫：`GoRouter.redirect` 函数（检查 JWT + 角色权限）
- 导航结构：`ShellRoute`（保持底部栏状态）+ 嵌套路由

4.2 路由树定义

```
// lib/router/app_router.dart

final appRouter = GoRouter(
  initialLocation: '/login',
  redirect: _authGuard,   // 全局守卫

  routes: [
    // ── 认证路由 ──
    GoRoute(path: '/login', builder: (ctx, s) => const LoginPage()),
    GoRoute(path: '/forgot-password', builder: (ctx, s) => const
ForgotPasswordPage()),

    // ── 主导航骨架（底部 TabBar）──
    ShellRoute(
      builder: (ctx, s, child) => MainScaffold(child: child),
      routes: [

        // ── Tab 1: 概览 Dashboard ──
        GoRoute(
          path: '/dashboard',
          builder: (ctx, s) => const DashboardPage(),    // NOI + 出租率 +
WALE 汇总
          routes: [
```

```
            GoRoute(path: 'noi-detail', builder: (ctx, s) => const
NoiDetailPage()),
            GoRoute(path: 'wale-detail', builder: (ctx, s) => const
WaleDetailPage()),
            GoRoute(path: 'kpi',        builder: (ctx, s) => const
KpiDashboardPage()),
            GoRoute(
              path: 'kpi/scheme/:schemeId',
              builder: (ctx, s) => KpiSchemeDetailPage(schemeId:
s.pathParameters['schemeId']!),
            ),
          ],
        ),

        // ── Tab 2: 资产 Assets ──
        GoRoute(
          path: '/assets',
          builder: (ctx, s) => const AssetOverviewPage(),   // 三业态出租率看板
          routes: [
            GoRoute(
              path: 'building/:buildingId',
              builder: (ctx, s) => BuildingDetailPage(id:
s.pathParameters['buildingId']!),
              routes: [
                GoRoute(
                  path: 'floor/:floorId',
                  builder: (ctx, s) => FloorMapPage(           // CAD 热区图
                    buildingId: s.pathParameters['buildingId']!,
                    floorId: s.pathParameters['floorId']!,
                  ),
                  routes: [
                    GoRoute(
                      path: 'unit/:unitId',
                      builder: (ctx, s) => UnitDetailPage(unitId:
s.pathParameters['unitId']!),
                      routes: [
                        GoRoute(path: 'renovation/add',   builder: (ctx, s)
=> RenovationFormPage(unitId: s.pathParameters['unitId']!)),
                        GoRoute(path: 'renovation/:rid',  builder: (ctx, s)
=> RenovationDetailPage(id: s.pathParameters['rid']!)),
                      ],
                    ),
                  ],
                ),
              ],
            ),
            GoRoute(path: 'import', builder: (ctx, s) => const
UnitImportPage()),   // Excel 批量导入
          ],
        ),

        // ── Tab 3: 租务 Contracts ──
        GoRoute(
          path: '/contracts',
```

```
          builder: (ctx, s) => const ContractListPage(),
          routes: [
            GoRoute(path: 'new',        builder: (ctx, s) => const
ContractFormPage()),
            GoRoute(
              path: ':contractId',
              builder: (ctx, s) => ContractDetailPage(id:
s.pathParameters['contractId']!),
              routes: [
                GoRoute(path: 'edit',          builder: (ctx, s) =>
ContractEditPage(id: s.pathParameters['contractId']!)),
                GoRoute(path: 'escalation',    builder: (ctx, s) =>
EscalationConfigPage(contractId: s.pathParameters['contractId']!)),
                GoRoute(path: 'subleases',     builder: (ctx, s) =>
SubleaseListPage(masterContractId: s.pathParameters['contractId']!)),
                GoRoute(path: 'subleases/new', builder: (ctx, s) =>
SubleaseFormPage(masterContractId: s.pathParameters['contractId']!)),
                GoRoute(path: 'renew',         builder: (ctx, s) =>
ContractRenewPage(parentId: s.pathParameters['contractId']!)),
              ],
            ),
          ],
        ),

        GoRoute(
          path: '/tenants',
          builder: (ctx, s) => const TenantListPage(),
          routes: [
            GoRoute(path: 'new',      builder: (ctx, s) => const
TenantFormPage()),
            GoRoute(path: ':tenantId', builder: (ctx, s) =>
TenantDetailPage(id: s.pathParameters['tenantId']!)),
          ],
        ),

        // ── Tab 4: 财务 Finance ──
        GoRoute(
          path: '/finance',
          builder: (ctx, s) => const FinanceOverviewPage(),    // NOI 看板入
口
          routes: [
            GoRoute(
              path: 'invoices',
              builder: (ctx, s) => const InvoiceListPage(),
              routes: [
                GoRoute(path: ':invoiceId',      builder: (ctx, s) =>
InvoiceDetailPage(id: s.pathParameters['invoiceId']!)),
                GoRoute(path: ':invoiceId/pay',  builder: (ctx, s) =>
PaymentFormPage(invoiceId: s.pathParameters['invoiceId']!)),
              ],
            ),
            GoRoute(path: 'expenses',     builder: (ctx, s) => const
ExpenseListPage()),
```

```
            GoRoute(path: 'expenses/new', builder: (ctx, s) => const
ExpenseFormPage()),
          ],
        ),

        // ── Tab 5: 工单 Workorders ──
        GoRoute(
          path: '/workorders',
          builder: (ctx, s) => const WorkOrderListPage(),
          routes: [
            GoRoute(path: 'new', builder: (ctx, s) => const
WorkOrderFormPage()),     // 移动端报修入口
            GoRoute(path: 'scan', builder: (ctx, s) => const QrScanPage()),
// 扫码报修
            GoRoute(
              path: ':orderId',
              builder: (ctx, s) => WorkOrderDetailPage(id:
s.pathParameters['orderId']!),
              routes: [
                GoRoute(path: 'approve',  builder: (ctx, s) =>
WorkOrderApprovePage(id: s.pathParameters['orderId']!)),
                GoRoute(path: 'complete', builder: (ctx, s) =>
WorkOrderCompletePage(id: s.pathParameters['orderId']!)),
              ],
            ),
          ],
        ),

      ],
    ),

    // ─── 二房东独立入口（脱离主导航骨架）───
    GoRoute(
      path: '/sublease-portal',
      builder: (ctx, s) => const SubLandlordPortalPage(),    // 独立简洁界面
      routes: [
        GoRoute(path: 'units',         builder: (ctx, s) => const
SubLandlordUnitListPage()),
        GoRoute(path: 'units/:unitId/fill', builder: (ctx, s) =>
SubleaseFillingPage(unitId: s.pathParameters['unitId']!)),
        GoRoute(path: 'import',        builder: (ctx, s) => const
SubleaseImportPage()),
      ],
    ),

    // ─── 系统设置（仅 super_admin / ops_manager）───
    GoRoute(
      path: '/settings',
      builder: (ctx, s) => const SettingsPage(),
      routes: [
        GoRoute(path: 'users',        builder: (ctx, s) => const
UserManagementPage()),
        GoRoute(path: 'users/new',    builder: (ctx, s) => const
UserFormPage()),
```

```
        GoRoute(path: 'kpi/schemes',  builder: (ctx, s) => const
KpiSchemeListPage()),
        GoRoute(path: 'kpi/schemes/new', builder: (ctx, s) => const
KpiSchemeFormPage()),
        GoRoute(path: 'escalation/templates', builder: (ctx, s) => const
EscalationTemplateListPage()),
        GoRoute(path: 'alerts',       builder: (ctx, s) => const
AlertCenterPage()),
      ],
    ),
  ],
);
```

4.3 路由守卫（角色路由映射）

```
// lib/router/auth_guard.dart

String? _authGuard(BuildContext context, GoRouterState state) {
  final authState = context.read<AuthBloc>().state;

  // 未登录 → 跳登录页
  if (authState is AuthUnauthenticated) {
    if (state.matchedLocation == '/login') return null;
    return '/login';
  }

  final role = authState.user.role;
  final path = state.matchedLocation;

  // 二房东只能访问 sublease-portal 下的路由
  if (role == UserRole.subLandlord && !path.startsWith('/sublease-portal'))
{
    return '/sublease-portal';
  }

  // 前线员工只能访问工单、只读资产
  if (role == UserRole.frontline) {
    final allowed = ['/dashboard', '/workorders', '/assets'];
    if (!allowed.any((p) => path.startsWith(p))) return '/workorders';
  }

  // 财务只能访问财务模块 + 概览
  if (role == UserRole.financeStaff) {
    final allowed = ['/dashboard', '/finance', '/contracts', '/tenants'];
    if (!allowed.any((p) => path.startsWith(p))) return '/finance';
  }

  return null; // 放行
}
```

---

5. RBAC 权限矩阵代码实现

5.1 权限枚举定义

```
// lib/core/rbac/permissions.dart

enum Permission {
  // 资产模块
  assetsRead,
  assetsWrite,
  assetsImport,
  cadUpload,

  // 租务模块
  contractsRead,
  contractsWrite,
  tenantsRead,
  tenantsWrite,
  tenantsViewFullCert,   // 查看完整证件号（需二次授权）

  // 财务模块
  financeRead,
  financeWrite,
  invoiceVerify,         // 账单核销
  noiBuild Read,

  // 工单模块
  workOrderCreate,
  workOrderRead,
  workOrderApprove,
  workOrderComplete,

  // 二房东穿透
  subleaseRead,          // 查看穿透看板
  subleaseWrite,         // 内部人工录入/审核
  subleasePortalAccess,  // 二房东自助录入入口

  // 系统管理
  userManage,
  kpiSchemeManage,
  kpiSchemeView,
  auditLogRead,
}
```

5.2 角色-权限矩阵

```
// lib/core/rbac/role_permissions.dart

const Map<UserRole, Set<Permission>> rolePermissions = {
  UserRole.superAdmin: {
    ...Permission.values,  // 拥有全部权限
```

```
},

UserRole.opsManager: {
  Permission.assetsRead,
  Permission.assetsWrite,
  Permission.cadUpload,
  Permission.contractsRead,
  Permission.contractsWrite,
  Permission.tenantsRead,
  Permission.tenantsWrite,
  Permission.financeRead,
  Permission.noiRead,
  Permission.workOrderRead,
  Permission.workOrderApprove,
  Permission.subleaseRead,
  Permission.subleaseWrite,
  Permission.kpiSchemeManage,
  Permission.kpiSchemeView,
  Permission.auditLogRead,
},

UserRole.leasingAgent: {
  Permission.assetsRead,
  Permission.contractsRead,
  Permission.contractsWrite,
  Permission.tenantsRead,
  Permission.tenantsWrite,
  Permission.financeRead,     // 只读，用于查看账单
  Permission.subleaseRead,
  Permission.subleaseWrite,
  Permission.kpiSchemeView,
},

UserRole.financeStaff: {
  Permission.assetsRead,
  Permission.contractsRead,
  Permission.tenantsRead,
  Permission.financeRead,
  Permission.financeWrite,
  Permission.invoiceVerify,
  Permission.noiRead,
  Permission.kpiSchemeView,
},

UserRole.frontline: {
  Permission.assetsRead,
  Permission.tenantsRead,     // 只读，不含证件号
  Permission.workOrderCreate,
  Permission.workOrderRead,
  Permission.workOrderComplete,
},

UserRole.subLandlord: {
  Permission.subleasePortalAccess,
```

```
    // 数据范围由行级隔离控制，不在此处扩展
  },
};
```

5.3 RBAC 中间件（Shelf）

```
// lib/core/middleware/rbac_middleware.dart

import 'package:shelf/shelf.dart';
import '../rbac/role_permissions.dart';
import '../request_context.dart';

/// 路由级别权限守卫装饰器
///
/// 使用方式:
///   router.get('/api/contracts', rbacGuard(Permission.contractsRead)
(handler));
Middleware rbacGuard(Permission required) {
  return (Handler innerHandler) {
    return (Request request) async {
      final ctx = RequestContext.of(request);

      if (ctx == null) {
        return Response(401, body: '{"error":"unauthenticated"}',
            headers: {'Content-Type': 'application/json'});
      }

      final perms = rolePermissions[ctx.role] ?? {};
      if (!perms.contains(required)) {
        return Response(403, body: '{"error":"forbidden"}',
            headers: {'Content-Type': 'application/json'});
      }

      return innerHandler(request);
    };
  };
}

/// 多权限 OR 检查（拥有其中任一即放行）
Middleware rbacGuardAny(Set<Permission> anyOf) {
  return (Handler innerHandler) {
    return (Request request) async {
      final ctx = RequestContext.of(request);
      if (ctx == null) return Response(401);

      final perms = rolePermissions[ctx.role] ?? {};
      if (perms.intersection(anyOf).isEmpty) return Response(403);

      return innerHandler(request);
    };
  };
}
```

5.4 请求上下文注入（JWT 解析中间件）

```
// lib/core/middleware/auth_middleware.dart

Middleware authMiddleware(TokenService tokenService) {
  return (Handler inner) {
    return (Request request) async {
      // 公开路由白名单（跳过验证）
      const publicPaths = ['/api/auth/login', '/api/auth/refresh'];
      if (publicPaths.any((p) => request.url.path == p)) {
        return inner(request);
      }

      final authHeader = request.headers['authorization'] ?? '';
      if (!authHeader.startsWith('Bearer ')) return Response(401);

      final token = authHeader.substring(7);
      final claims = tokenService.verify(token);
      if (claims == null) return Response(401);

      // 将上下文注入到请求中（通过 Request.change context）
      final ctx = RequestContext(
        userId: claims['sub'] as String,
        role: UserRole.values.byName(claims['role'] as String),
        masterContractId: claims['master_contract_id'] as String?, // 二房东
专用
      );

      return inner(request.change(context: {RequestContext.key: ctx}));
    };
  };
}
```

5.5 路由注册示例（Controller 层）

```
// lib/modules/contracts/controllers/contract_controller.dart

Router contractRoutes() {
  final router = Router();
  final service = ContractService(ContractRepository());

  router.get('/api/contracts',
      rbacGuard(Permission.contractsRead)(
        (req) => service.listContracts(RequestContext.of(req)!),
      ));

  router.post('/api/contracts',
      rbacGuard(Permission.contractsWrite)(
        (req) => service.create(req, RequestContext.of(req)!),
      ));

  router.get('/api/contracts/<id>',
      rbacGuard(Permission.contractsRead)(
```

```
        (req, id) => service.getById(id, RequestContext.of(req)!),
      ));

  // 查看完整证件号：需要额外权限
  router.get('/api/tenants/<id>/cert',
      rbacGuard(Permission.tenantsViewFullCert)(
        (req, id) => service.getFullCert(id, RequestContext.of(req)!),
      ));

  return router;
}
```

---

6. 二房东数据行级隔离 Repository 实现

6.1 设计原则

行级隔离在 **Repository** 层强制实施，Service 层通过传递 `RequestContext` 触发，不依赖调用方主动传参，消除遗漏隔离的风险。

角色	隔离范围	实现机制
内部所有角色	无数据范围限制	SQL 无附加 WHERE 条件
`sub_landlord`	仅限 `master_contract_id` 对应的单元和子租赁	JWT Claims 携带 `master_contract_id`；Repository 层强制附加 `AND master_contract_id = $scope`

6.2 RequestContext 结构

```
// lib/core/request_context.dart

@freezed
class RequestContext with _$RequestContext {
  const factory RequestContext({
    required String userId,
    required UserRole role,
    String? masterContractId,  // 非 null 当且仅当 role = sub_landlord
  }) = _RequestContext;

  /// 是否为二房东角色（需要行级隔离）
  bool get isSubLandlord =>
      role == UserRole.subLandlord && masterContractId != null;

  static const key = 'request_context';
  static RequestContext? of(Request request) =>
      request.context[key] as RequestContext?;
}
```

6.3 SubleaseRepository 行级隔离实现

```
// lib/modules/subleases/repositories/sublease_repository.dart

class SubleaseRepository {
  final Connection _db;

  SubleaseRepository(this._db);

  /// 查询子租赁列表
  /// 若 ctx.isSubLandlord == true，自动附加 master_contract_id 过滤
  Future<List<SubLease>> findAll(RequestContext ctx, {
    ReviewStatus? reviewStatus,
    int limit = 50,
    int offset = 0,
  }) async {
    final (scopeClause, params) = _buildScopeClause(ctx);

    var idx = params.length + 1;
    final conditions = [scopeClause];
    if (reviewStatus != null) {
      conditions.add('review_status = \$$idx');
      params.add(reviewStatus.name);
      idx++;
    }

    final where = conditions.join(' AND ');
    final sql = '''
      SELECT s.*, u.unit_no, u.property_type,
             c.contract_no AS master_contract_no
      FROM   subleases s
      JOIN   units u ON u.id = s.unit_id
      JOIN   contracts c ON c.id = s.master_contract_id
      WHERE  $where
      ORDER  BY s.created_at DESC
      LIMIT  \$$idx OFFSET \$${idx + 1}
    ''';
    params.addAll([limit, offset]);

    final rows = await _db.execute(sql, parameters: params);
    return rows.map(SubLease.fromRow).toList();
  }

  /// 查询单条子租赁（带所有权验证）
  Future<SubLease?> findById(String id, RequestContext ctx) async {
    final (scopeClause, params) = _buildScopeClause(ctx);
    params.add(id);

    final sql = '''
      SELECT * FROM subleases
      WHERE  $scopeClause AND id = \$${params.length}
    ''';
    final rows = await _db.execute(sql, parameters: params);
    return rows.isEmpty ? null : SubLease.fromRow(rows.first);
```

```
  }

  /// 新增子租赁（二房东只能在自身主合同范围内的单元创建）
  Future<SubLease> create(CreateSubleaseDto dto, RequestContext ctx) async {
    // 若为二房东，强制 master_contract_id 必须与 JWT 中一致
    if (ctx.isSubLandlord &&
        dto.masterContractId != ctx.masterContractId) {
      throw ForbiddenException('跨主合同操作被拒绝');
    }

    // 验证单元是否在主合同覆盖范围内
    await _assertUnitInScope(dto.unitId, dto.masterContractId);

    const sql = '''
      INSERT INTO subleases
        (master_contract_id, unit_id, sub_tenant_name, sub_tenant_type,
         contact_name, contact_phone_encrypted, contact_phone_hint,
         cert_no_encrypted, cert_no_hint,
         start_date, end_date, monthly_rent,
         occupancy_status, occupant_count, notes,
         review_status, submitted_by)
      VALUES (\$1,\$2,\$3,\$4,\$5,\$6,\$7,\$8,\$9,\$10,\$11,\$12,\$13,\$14,\
$15,
              \$16,\$17)
      RETURNING *
    ''';

    final reviewStatus = ctx.isSubLandlord
        ? ReviewStatus.pendingReview   // 二房东提交后进入待审核
        : ReviewStatus.approved;       // 内部人工录入直接生效

    final params = [
      dto.masterContractId,
      dto.unitId,
      dto.subTenantName,
      dto.subTenantType.name,
      dto.contactName,
      dto.contactPhoneEncrypted,       // 调用方需提前加密
      dto.contactPhoneHint,
      dto.certNoEncrypted,
      dto.certNoHint,
      dto.startDate.toIso8601String(),
      dto.endDate.toIso8601String(),
      dto.monthlyRent,
      dto.occupancyStatus.name,
      dto.occupantCount,
      dto.notes,
      reviewStatus.name,
      ctx.userId,
    ];

    final rows = await _db.execute(sql, parameters: params);
    return SubLease.fromRow(rows.first);
  }
```

```
  /// 更新子租赁（带所有权验证）
  Future<SubLease> update(String id, UpdateSubleaseDto dto, RequestContext
ctx) async {
    // 先查询验证所有权
    final existing = await findById(id, ctx);
    if (existing == null) throw NotFoundException('子租赁记录不存在或无权访问');

    // 二房东只能修改 draft / rejected 状态的记录
    if (ctx.isSubLandlord &&
        existing.reviewStatus != ReviewStatus.draft &&
        existing.reviewStatus != ReviewStatus.rejected) {
      throw ForbiddenException('已提交审核的记录不可修改');
    }

    const sql = '''
      UPDATE subleases SET
        sub_tenant_name = COALESCE(\$2, sub_tenant_name),
        monthly_rent    = COALESCE(\$3, monthly_rent),
        occupancy_status = COALESCE(\$4, occupancy_status),
        notes           = COALESCE(\$5, notes),
        review_status   = \$6,
        updated_at      = NOW()
      WHERE id = \$1
      RETURNING *
    ''';
    final rows = await _db.execute(sql, parameters: [
      id, dto.subTenantName, dto.monthlyRent,
      dto.occupancyStatus?.name, dto.notes,
      ctx.isSubLandlord ? ReviewStatus.draft.name :
ReviewStatus.approved.name,
    ]);
    return SubLease.fromRow(rows.first);
  }

  // ── 私有辅助方法 ──────────────────────────────────────────

  /// 构建范围 WHERE 子句
  /// - 内部角色：无附加条件（返回恒真条件）
  /// - 二房东：强制附加 master_contract_id = $n
  (String clause, List<Object?> params) _buildScopeClause(RequestContext
ctx) {
    if (ctx.isSubLandlord) {
      return ('master_contract_id = \$1', [ctx.masterContractId]);
    }
    return ('TRUE', []);
  }

  /// 验证 unitId 是否在主合同覆盖范围内
  Future<void> _assertUnitInScope(String unitId, String masterContractId)
async {
    const sql = '''
      SELECT 1 FROM contract_units
```

```
      WHERE  contract_id = \$1 AND unit_id = \$2
    ''';
    final rows = await _db.execute(sql, parameters: [masterContractId,
unitId]);
    if (rows.isEmpty) throw BadRequestException('指定单元不在主合同覆盖范围内');
  }
}
```

6.4 审计日志自动记录

所有子租赁写操作结束后，Service 层调用公共审计方法：

```
// lib/modules/subleases/services/sublease_service.dart

class SubleaseService {
  final SubleaseRepository _repo;
  final AuditLogRepository _audit;

  Future<SubLease> createSublease(CreateSubleaseDto dto, RequestContext ctx)
async {
    final sublease = await _repo.create(dto, ctx);

    await _audit.log(
      operatorId: ctx.userId,
      module: 'subleases',
      action: 'create',
      entityType: 'sublease',
      entityId: sublease.id,
      newValue: sublease.toJson(),
    );

    return sublease;
  }
}
```

6.5 二房东账号生命周期管理

```
-- 主合同到期后自动冻结二房东账号（数据库触发器）
CREATE OR REPLACE FUNCTION freeze_sub_landlord_on_contract_expiry()
RETURNS TRIGGER AS $$
BEGIN
  IF NEW.status IN ('expired', 'terminated') AND OLD.status != NEW.status
THEN
    UPDATE users
       SET is_active = FALSE
     WHERE role = 'sub_landlord'
       AND master_contract_id = NEW.id;
  END IF;
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_freeze_sub_landlord
```

```
  AFTER UPDATE OF status ON contracts
  FOR EACH ROW
  EXECUTE FUNCTION freeze_sub_landlord_on_contract_expiry();
```

```
-- 登录失败锁定与会话版本控制由 users 表字段配合 AuthService 实现
-- failed_login_attempts >= 5 时写入 locked_until = NOW() + interval '30
minutes'
-- session_version 每次改密、强制登出或主合同冻结后 +1，旧 JWT 自动失效
```

---

7. 关键数据流说明

7.1 账单自动生成流程

```
定时任务（每日凌晨1点）
  └─ AlertService.checkExpiringContracts()
       └─ 查询 end_date ≤ NOW() + 90d 的合同
            └─ 写入 alerts（expiry_90d / expiry_60d / expiry_30d）

定时任务（每账期前 N 天）
  └─ InvoiceService.generatePeriodInvoices()
       └─ 遍历所有 active 合同
            └─ RentEscalationService.computeRentForPeriod(contract, period)
                 └─ 按阶段规则计算应收租金
            └─ 写入 invoices（rent + mgmt_fee + utility）
      └─ 写入 invoice_items（按费项拆分）

收款入账
  └─ PaymentController.createReceipt()
    └─ ReceivableService.allocate(payment, targetInvoices)
      ├─ 默认按"先到期先核销"生成 payment_allocations
      ├─ 支持单笔收款分配到多张 invoice
      └─ 回写 invoice.paid_amount / outstanding_amount / status
```

7.2 WALE 实时计算

```
GET /api/contracts/wale?groupBy=property_type

ContractController
  └─ WaleService.compute(groupBy: 'property_type', ctx)
       └─ SQL:
            SELECT
              property_type,
              SUM(
                EXTRACT(EPOCH FROM (c.end_date - NOW())) / 86400 / 365
                * RentEscalationService.annualizedRent(c)
              ) / SUM(RentEscalationService.annualizedRent(c)) AS wale
            FROM contracts c
            WHERE c.status = 'active'
```

```
            GROUP BY property_type
```

7.3 KPI 自动打分（线性插值）

> **Phase 1 约束**: 本流程输出的是试运行评分结果，快照生成后默认冻结；补录或口径调整需显式触发重算，并保留审计日志。

```
double _interpolateScore(double actual, double fullThreshold, double
passThreshold) {
  if (actual >= fullThreshold) return 100.0;
  if (actual < passThreshold) {
    // 低于及格线：按比例 0~60
    return (actual / passThreshold * 60).clamp(0, 60);
  }
  // 及格线~满分线：线性插值 60~100
  return 60 + (actual - passThreshold) /
      (fullThreshold - passThreshold) * 40;
}
```

7.4 任务可靠性与人工补偿

```
JobRunner.execute(job)
  ├─ 写入 job_execution_log(status=running)
  ├─ 调用业务任务（预警、催收、提醒、导入后处理）
  ├─ 成功 → 标记 success
  └─ 失败 → 记录 error_message / retry_count
            ├─ RetryScheduler 按退避策略自动重试
            └─ 后台失败任务列表支持人工重跑或补发
```

文档结束